

Documento Técnico: Integración, Flujos y Validaciones del Frontend — Proyecto HeartCoin

Introducción

Este documento describe cómo la aplicación cliente de HeartCoin se integra con las APIs del backend, documenta los flujos de usuario efectivamente implementados de principio a fin, y detalla los mecanismos de validación aplicados en los formularios de la aplicación. Consolida tres actividades relacionadas —integración del frontend con las APIs del backend, implementación de flujos de usuario, e implementación de validaciones en formularios— por describir, en conjunto, cómo la interfaz traduce la interacción del usuario en operaciones concretas contra el backend.

Objetivo

Documentar el patrón de integración del cliente con el backend, los flujos de usuario completos implementados en la aplicación, y las reglas de validación aplicadas antes de que cualquier dato llegue al servidor.

Alcance

Este documento cubre el patrón de comunicación entre los servicios del cliente y los mecanismos de API descritos en `APIS_HEARTCOIN.md`, los flujos de usuario de mayor relevancia del sistema (autenticación, verificación de correo, canje de HC, interacción social), y las validaciones de formulario observadas en las pantallas correspondientes. No cubre la correspondencia de estos flujos con maquetas de diseño (Figma), por no haberse tenido acceso al material visual de referencia.

Metodología

La información se obtuvo mediante inspección directa del código fuente de los servicios y pantallas del frontend, y se contrastó contra el comportamiento real del backend ya documentado y verificado en `APIS_HEARTCOIN.md` y `FLUJOS_TRANSACCIONALES.md`.

Desarrollo del Análisis

1. Integración del Frontend con las APIs del Backend

La integración se realiza en su totalidad a través del SDK oficial `supabase_flutter`, sin llamadas HTTP manuales salvo en los casos que requieren `multipart/form-data` (subida de archivos). Cada servicio del frontend traduce una operación de negocio a una de las cuatro formas de comunicación disponibles:

Forma de comunicación	Ejemplo de uso en el frontend
Lectura/escritura directa (PostgREST)	<code>WalletService</code> leyendo <code>hc_transactions</code> ; <code>FollowService</code> insertando en <code>follows</code>
Invocación de función RPC	<code>CheckinService</code> invocando <code>checkin_iniciativa</code> ; <code>BeneficioService</code> invocando <code>redeem_beneficio</code>
Suscripción en tiempo real	<code>WalletService.profileStream()</code> , feed de publicaciones, contador de notificaciones no leídas
Invocación de Edge Function	<code>OriginalAuthService</code> , <code>AccountService</code> , <code>MediaService</code>

Patrón de manejo de respuesta uniforme: los servicios que invocan Edge Functions siguen un patrón consistente: verifican el código de estado HTTP de la respuesta, y capturan explícitamente `FunctionException` (el tipo de excepción que el SDK de Supabase lanza ante respuestas no exitosas) además de sus propias excepciones tipadas — un ajuste incorporado tras detectar, durante pruebas reales, que omitir esta captura específica resultaba en mensajes de error genéricos que ocultaban la causa real de una falla.

2. Flujos de Usuario Implementados

Se documentan los flujos de mayor relevancia funcional del sistema, verificados como operativos de principio a fin:

Registro con verificación de correo:

Formulario de registro → creación de cuenta (Supabase) → disparo de verificación (Original Auth) → pantalla de código → confirmación → acceso a la aplicación

La aplicación **bloquea el acceso** hasta completar la verificación, como decisión de producto explícita.

Inicio de sesión y recuperación de contraseña:

Login directo (correo + contraseña) → sesión activa

"Olvidé mi contraseña" → correo → código de 6 dígitos → nueva contraseña → sesión cerrada → login con la nueva contraseña

Canje de HC (beneficio o servicio):

Escaneo de código QR → identificación automática del tipo (beneficio o servicio) → validación de reglas de negocio en el servidor → confirmación visual del canje

El escáner de la billetera implementa una resolución automática de tipo: intenta primero interpretar el código como beneficio, y si no corresponde, como servicio, sin que el usuario deba indicar de antemano qué está escaneando.

Interacción social (bloqueo de usuario):

Menú contextual sobre una publicación o perfil → confirmación → bloqueo aplicado → publicaciones del usuario bloqueado desaparecen del feed de forma inmediata (filtrado local) → persistencia del bloqueo confirmada en el servidor

Este flujo aplica un patrón de **actualización optimista**: el efecto visual (desaparición del contenido) ocurre antes de confirmar la respuesta del servidor, revirtiéndose solo si la operación falla — un patrón repetido de forma consistente en otras interacciones sociales del sistema (likes, seguimiento).

Eliminación de cuenta:

Configuración → Eliminar cuenta → advertencia + checklist de lo que se borra → confirmación obligatoria (casilla) → ejecución → cierre de sesión → retorno a pantalla de login

3. Validaciones en Formularios

Las validaciones se aplican en dos niveles complementarios, nunca como sustituto uno del otro:

- **Validación de experiencia de usuario (cliente):** verificaciones inmediatas antes de intentar la operación de red, para dar retroalimentación instantánea sin esperar respuesta del servidor.
- **Validación de integridad (servidor):** las mismas reglas —y otras adicionales no verificables del lado del cliente— se aplican nuevamente en el backend (constraints de base de datos, funciones SECURITY DEFINER), de modo que ninguna regla de negocio depende exclusivamente de que el cliente la respete.

Formulario	Validaciones de cliente observadas
Registro	Correo y contraseña no vacíos; contraseña de al menos 6 caracteres; campos obligatorios por rol (nombre completo, país, ciudad, o los equivalentes de organización/empresa)
Recuperación de contraseña	Formato de correo válido (contiene @); código de verificación no vacío; nueva contraseña de al menos 6 caracteres; coincidencia entre contraseña y confirmación

Verificación de correo (Original Auth)	Código no vacío (sin asumir un formato o longitud fija, dado que el proveedor externo no garantiza un formato único)
Eliminación de cuenta	Casilla de confirmación obligatoria antes de habilitar el botón de acción

Ejemplo de corrección basada en comportamiento real observado: el campo de país en el formulario de registro se implementó originalmente como texto libre; al integrarse con Original Auth, se detectó que ese formato no era compatible con lo que el servicio externo esperaba, resolviéndose mediante el ajuste de qué datos se envían a esa integración específica, sin necesidad de modificar el formulario ni restringir la experiencia del usuario dentro de la aplicación.

Problemas Detectados

1. **Duplicación manual de reglas de validación entre cliente y servidor:** al no existir un esquema de validación compartido (por ejemplo, generado a partir de un contrato único), cada regla debe mantenerse sincronizada manualmente en ambos lados, con el riesgo de divergencia si una se actualiza sin la otra.
2. **El flujo de registro no es reversible ante fallo parcial:** si la creación de la cuenta en Supabase tiene éxito pero el disparo de verificación de correo falla, no existe un mecanismo automático de reintento o compensación más allá de que el usuario permanezca en la pantalla de verificación con la opción manual de reenvío.
3. **Ausencia de retroalimentación diferenciada por tipo de error de red:** la mayoría de los flujos capturan errores de forma genérica ("no se pudo completar la acción"), sin distinguir explícitamente entre falta de conectividad y otros tipos de falla — hallazgo ya identificado en la auditoría de UX referida en `ESTRUCTURA_IMPLEMENTACION_FRONTEND.md`.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto en mantenibilidad**, materializable únicamente si una regla de negocio cambia y se actualiza en un solo lado del sistema.
- El riesgo del punto 2 es de **impacto operativo bajo**, dado que el estado resultante (cuenta creada, verificación pendiente) es recuperable manualmente por el usuario mediante el reenvío de código, sin pérdida de datos.
- El riesgo del punto 3 es de **impacto en experiencia de usuario**, no en integridad del sistema: el usuario puede no saber si conviene reintentar de inmediato o esperar conectividad.

Fortalezas Identificadas

- **Validación en dos niveles aplicada de forma consistente**, sin depender exclusivamente del cliente para ninguna regla de integridad crítica.
- **Patrón de actualización optimista** aplicado de forma repetible en interacciones sociales, mejorando la percepción de velocidad sin comprometer la consistencia final de los datos.
- **Resolución automática de contexto en el escáner de la billetera** (beneficio vs. servicio), reduciendo fricción para el usuario sin requerir una decisión previa.
- **Capacidad de ajustar la integración con un tercero sin modificar la experiencia del formulario** (caso del campo país), evidenciando una separación adecuada entre la capa de presentación y la capa de integración externa.

Áreas de Mejora

- Evaluar la incorporación de un esquema de validación compartido o generado, para reducir el riesgo de divergencia entre cliente y servidor.
- Diferenciar explícitamente los mensajes de error de conectividad de otros tipos de falla en los flujos principales.
- Definir un mecanismo de recordatorio o seguimiento para cuentas que queden en estado de verificación pendiente por tiempo prolongado.

Recomendaciones

1. Mantener el patrón de doble validación (cliente + servidor) como estándar no negociable para cualquier nuevo formulario, dado que es la base de la integridad de datos del sistema.
2. Incorporar diferenciación de errores de red en los próximos ciclos de mejora de experiencia de usuario, aprovechando que ya fue identificado como hallazgo concreto.
3. Documentar explícitamente, en cada nuevo flujo que involucre un servicio externo, qué validaciones de cliente existentes podrían requerir ajuste por incompatibilidad de formato — replicando la lección aprendida con el campo de país en la integración con Original Auth.

Conclusión

La integración del frontend de HeartCoin con su backend sigue un patrón consistente basado en el SDK oficial de Supabase, complementado por un manejo de errores cada vez más refinado a partir de la experiencia real de implementación. Los flujos de usuario documentados —particularmente el registro con verificación de correo y el canje de HC— están verificados como operativos de principio a fin, y las validaciones de formulario se aplican de forma disciplinada en dos capas complementarias. Las oportunidades de mejora identificadas son de naturaleza incremental y no comprometen la funcionalidad actual del sistema.